Министерство образования Российской
Федерации Пензенский государственный
университет Кафедра «Вычислительная
техника»

ОТЧЕТ

по лабораторной работе №7

по курсу «Логика и основы алгоритмизации в инженерных задачах» на тему «Обход графа в глубину»

Выполнил: ст. гр. 25ВВВ1
Какулин В.С.
Самсонов М.А.
Проверил: к.т.н. доцент
Юрова О.В.
к.т.н., ст.
преподователь Деев
М.В.

Пенза 2026

Название:

Обход графа в глубину

Цель работы:

Изучить алгоритм обхода графа в глубину (DFS), реализовать его в рекурсивной и нерекурсивной форме, а также для матричной и списочной форм представления графа.

Лабораторное задание:

Задание 1:

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.
3. Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2:

1. Для матричной формы представления графов выполните преобразование рекурсивной реализации обхода графа к не рекурсивной.

Ход работы:

Описание используемых алгоритмов

Обход графа в глубину — один из способов обхода графа, идея которого состоит в том, чтобы, начав обход из некоторой вершины, всегда переходить по первой встречающейся связи в следующую вершину, пока существует такая возможность. Как только возможности продвижения исчерпаны, алгоритм возвращается на один шаг назад и ищет следующий вариант продвижения. Чтобы не заходить повторно в уже пройденные вершины, они помечаются как посещённые.

- generate_adjacency_matrix(size, edge_percent): динамически выделяет матрицу size × size и заполняет её: для каждой пары i < j ребро добавляется с вероятностью edge_percent процентов (rand() % 100 < edge_percent), при этом G[i][j] = G[j][i], диагональ равна нулю.
- print_matrix(G, size): выводит матрицу смежности с заголовками строк и столбцов.

- dfs_recursive(v, size, G, NUM): рекурсивная реализация обхода в глубину. Помечает вершину v как посещённую (NUM[v] = true), выводит её номер, затем в цикле просматривает строку G[v] и для каждой смежной непосещённой вершины рекурсивно вызывает себя.
- run_task_1(size, G, start): обход в глубину по матрице смежности. Создаёт массив посещённых вершин, запускает dfs_recursive от стартовой вершины, а затем — от всех оставшихся непосещённых вершин. Замеряет время выполнения.
- run_task_2(size, G, start): нерекурсивная реализация обхода в глубину. Использует два стека: stack_v хранит вершины, stack_i — индекс текущей просматриваемой строки матрицы для каждой вершины. В цикле: если найдена смежная непосещённая вершина — она помечается, выводится и кладётся в стек; иначе вершина снимается со стека. Замеряет время выполнения.
- createNode / addEdge: вспомогательные функции построения списков смежности. addEdge добавляет вершину dest в список вершины src и наоборот.
- dfs_list_recursive(v, adjLists, NUM): рекурсивный обход в глубину по спискам смежности. Аналогичен dfs_recursive, но перебирает не строку матрицы, а связный список соседей.
- run_task_3(size, G, start): строит списки смежности из матрицы, выполняет обход DFS, выводит порядок вершин и освобождает память. Замеряет время выполнения.
- main: вводит число вершин и вероятность ребра, генерирует граф, выводит матрицу, вводит стартовую вершину и последовательно запускает задания 1–3, замеряя время работы каждого через clock().

Результат выполнения работы

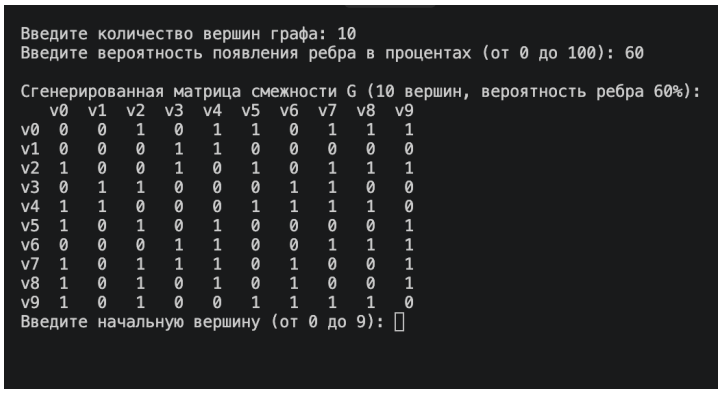


Рисунок 1 – стартовое окно при кол-ве вершин 10

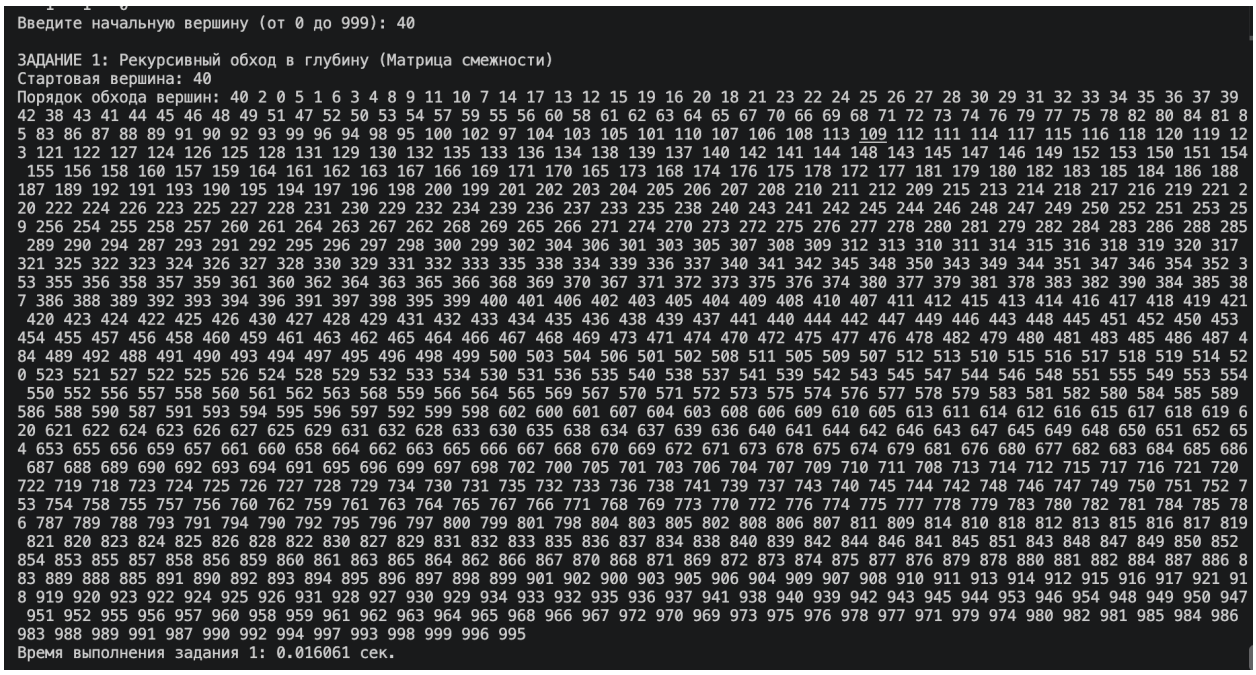


Рисунок 2 – результат выполнения программы задания 1. Тест на 1000 вершин.

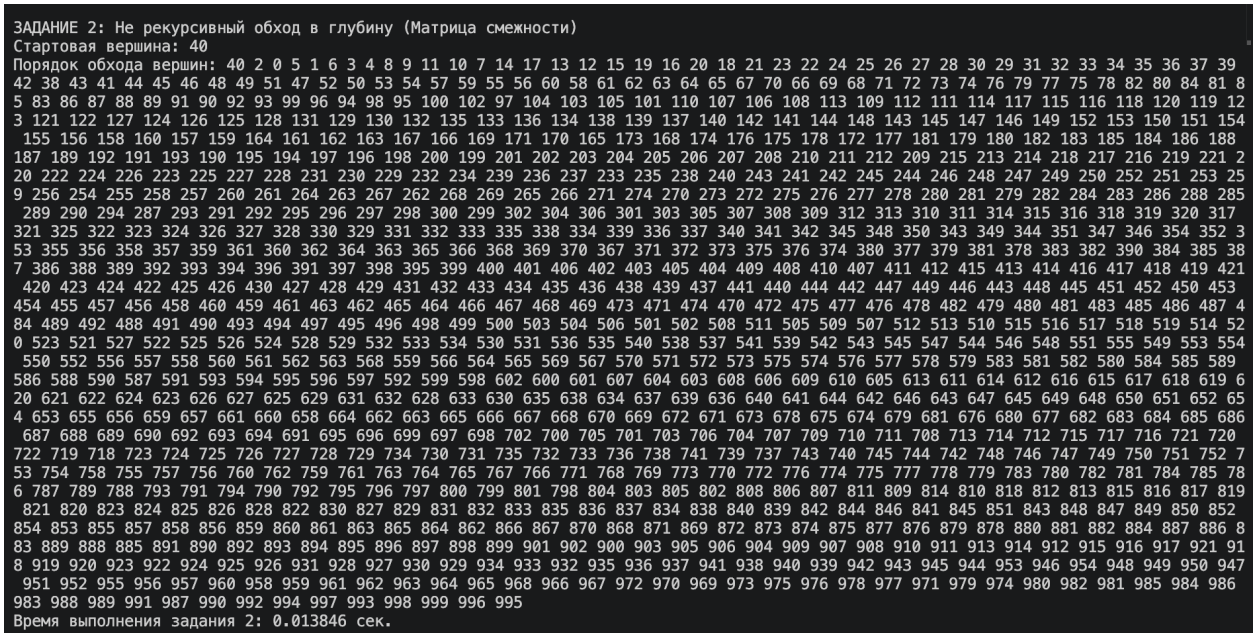


Рисунок 3 – результат выполнения программы задания 2. Тест на 1000 вершин.

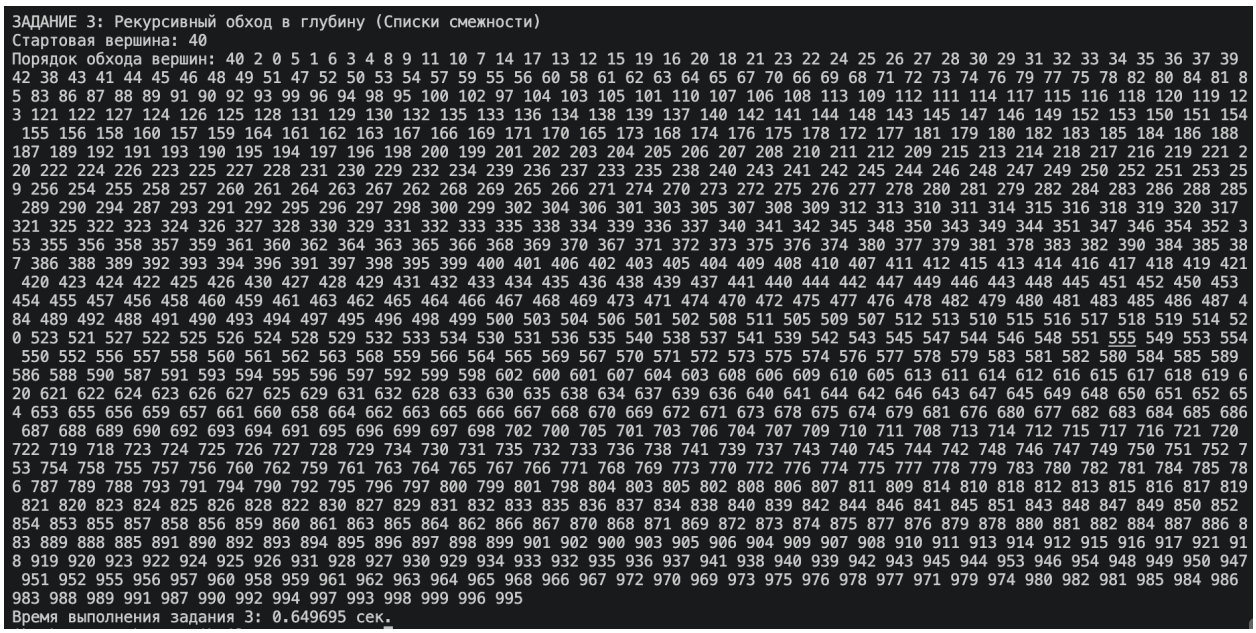


Рисунок 4 – результат выполнения программы задания 3. Тест на 1000 вершин.

Вывод:

В ходе выполнения лабораторной работы был изучен алгоритм обхода графа в глубину. Реализованы три варианта обхода: рекурсивный по матрице смежности, не рекурсивный по матрице смежности и рекурсивный по спискам смежности. Все реализации дают одинаковый порядок обхода вершин, корректно обрабатывают несвязные графы и работают с динамической памятью.